EXPERIMENT 10

Title:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Course Outcomes (LO):

Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Theory :

Hyperledger Fabric is an open-source, permissioned blockchain platform used to develop secure and distributed applications. It allows authorized organizations and participants to share information and perform transactions within a controlled network. It provides features such as identity management, access control, transaction verification and smart contract execution.

A Hyperledger Fabric network consists of components such as Peers, Ordering Service, Organizations and Channels. Peers maintain copies of the ledger and execute smart contracts, while the Ordering Service arranges transactions into blocks. Channels provide a private communication environment between participating organizations.

In Hyperledger Fabric, smart contracts are implemented using Chaincode, which defines the rules and business logic for managing assets. In this experiment, JavaScript-based `asset-transfer-basic` chaincode is deployed on the Fabric test network using Docker. A channel named `mychannel` is created to perform transactions.

The experiment demonstrates different asset management operations. The `InitLedger` function initializes the ledger with sample assets, `GetAllAssets` retrieves all assets, and `ReadAsset` fetches a specific asset. The `CreateAsset` function adds a new asset with attributes such as ID, colour, size, owner and value, while `TransferAsset` changes its ownership.

Thus, this experiment demonstrates how Hyperledger Fabric uses blockchain technology and chaincode to securely create, query and transfer digital assets across a distributed network.

Procedure:

Step 1: Update the System

Command:

sudo apt update
sudo apt upgrade -y

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32$ sudo apt update
[sudo] password for aarya_thorat:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1146 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [231 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.3 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1223 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [246 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [24.2 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1713 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [392 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [45.4 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
```

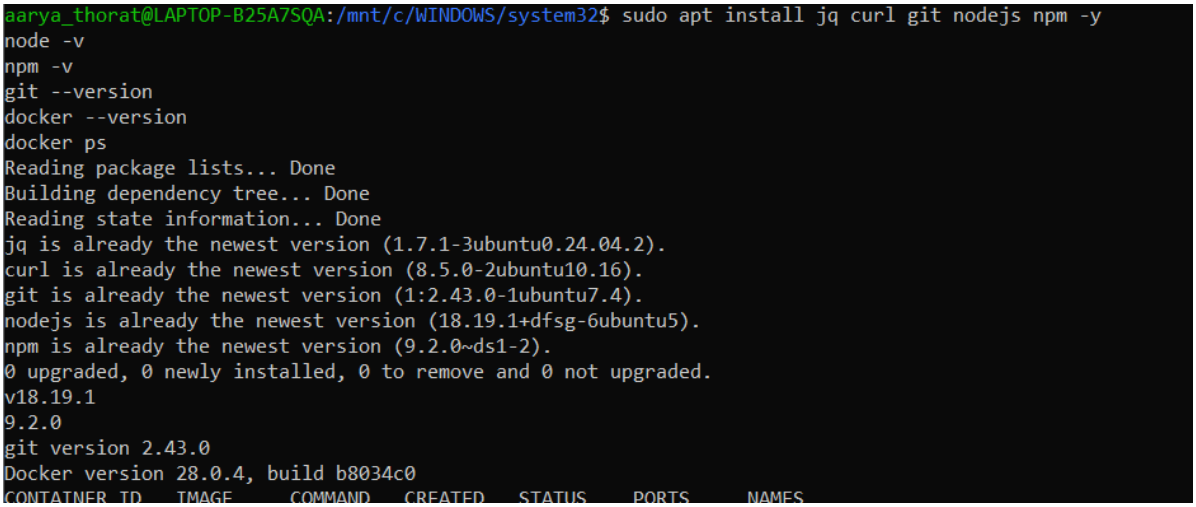
```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  apparmor ca-certificates cloud-init curl distro-info-data gir1.2-glib-2.0 git git-man krb5-locales
  libapparmor1 libaudit-common libaudit1 libbrotli1 libc-bin libc6 libcurl3t64-gnutls libcurl4t64
  libegl-mesa0 libevent-core-2.1-7t64 libexpat1 libfreetype6 libgbm1 libgl1-mesa-dri libglib2.0-0t64
  libglib2.0-bin libglib2.0-data libglx-mesa0 libgssapi-krb5-2 libk5crypto3 libkrb5-3 libkrb5support
  liblzma5 libnetplan1 libnss-systemd libpam-systemd libpciaccess0 libperl5.38t64 libpng16-16t64
  libpolkit-agent-1-0 libpolkit-gobject-1-0 libpython3.12-minimal libpython3.12-stdlib
  libpython3.12t64 librsvg2-2 librsvg2-common libsqlite3-0 libssl3t64 libsystemd-shared libsystemd0
  libudev1 libx11-6 libx11-data libx11-xcb1 libxml2 libxtst6 locales mesa-libgallium
  mesa-vulkan-drivers netplan-generator netplan.io openssh-client openssl perl perl-base
  perl-modules-5.38 polkitd python3-jwt python3-netplan python3-requests python3-urllib3 python3.12
```

Explanation:
This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps

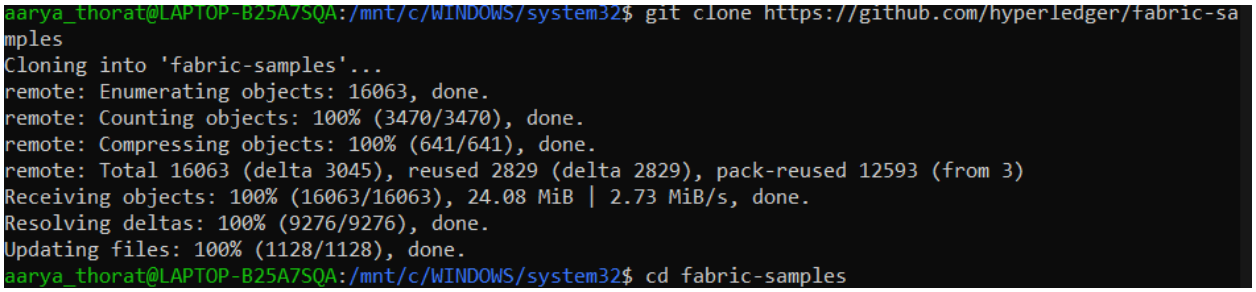


Explanation:
This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while docker ps ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples



Explanation:
This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples$ curl -sSLO https://raw.githubuserco
ntent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branch is intended to wor
k with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.16 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.16/hyperledger-fabric-li
nux-amd64-2.5.16.tar.gz
===> Will unpack to: /mnt/c/WINDOWS/system32/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  121M  100  121M    0     0  3681k      0  0:00:33  0:00:33 --:--:-- 3253k
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric
-ca-linux-amd64-1.5.17.tar.gz
===> Will unpack to: /mnt/c/WINDOWS/system32/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0      0      0 --:--:--  0:01:37 --:--:--     0
```

Explanation:
This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples$ docker images | grep fabric
hyperledger/fabric-peer             2.5       58979b65744b   4 months ago   232MB
hyperledger/fabric-peer             2.5.16    58979b65744b   4 months ago   232MB
hyperledger/fabric-peer             latest    58979b65744b   4 months ago   232MB
ghcr.io/hyperledger/fabric-peer     2.5.16    58979b65744b   4 months ago   232MB
hyperledger/fabric-orderer          2.5       1d918fc7e1e0   4 months ago   182MB
hyperledger/fabric-orderer          2.5.16    1d918fc7e1e0   4 months ago   182MB
hyperledger/fabric-orderer          latest    1d918fc7e1e0   4 months ago   182MB
ghcr.io/hyperledger/fabric-orderer  2.5.16    1d918fc7e1e0   4 months ago   182MB
hyperledger/fabric-ccenv            2.5       807493f095f6   4 months ago   1.01GB
hyperledger/fabric-ccenv            2.5.16    807493f095f6   4 months ago   1.01GB
hyperledger/fabric-ccenv            latest    807493f095f6   4 months ago   1.01GB
ghcr.io/hyperledger/fabric-ccenv    2.5.16    807493f095f6   4 months ago   1.01GB
hyperledger/fabric-baseos           2.5       109b4a15b282   4 months ago   257MB
hyperledger/fabric-baseos           2.5.16    109b4a15b282   4 months ago   257MB
hyperledger/fabric-baseos           latest    109b4a15b282   4 months ago   257MB
ghcr.io/hyperledger/fabric-baseos   2.5.16    109b4a15b282   4 months ago   257MB
hyperledger/fabric-ca               1.5       453a0a727e82   8 months ago   381MB
hyperledger/fabric-ca               1.5.17    453a0a727e82   8 months ago   381MB
hyperledger/fabric-ca               latest    453a0a727e82   8 months ago   381MB
ghcr.io/hyperledger/fabric-ca       1.5.17    453a0a727e82   8 months ago   381MB
```

Explanation:
This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples$ export PATH="$PWD/bin:$PATH"
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples$ export FABRIC_CFG_PATH="$PWD/config
"
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples$ peer version
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Explanation:
This step sets environment variables required to run Fabric CLI commands. The PATH variable

allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running peer version verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

cd ~/fabric-samples/test-network
./network.sh up createChannel

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ cd ~/fabric-samples/te
st-network
./network.sh up createChannel
-bash: cd: /home/aarya_thorat/fabric-samples/test-network: No such file or directory
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and usin
g database 'leveldb with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
/mnt/c/WINDOWS/system32/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organization
s
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 7/7
 ✔Network fabric_test                      Created                                          0.1s
 ✔Volume "compose_peer0.org2.example.com"  Created                                          0.0s
 ✔Volume "compose_orderer.example.com"     Created                                          0.0s
 ✔Volume "compose_peer0.org1.example.com"  Created                                          0.0s
 ✔Container peer0.org1.example.com         Started                                          1.2s
 ✔Container peer0.org2.example.com         Started                                          1.2s
 ✔Container orderer.example.com            Started                                          1.3s
CONTAINER ID   IMAGE                               COMMAND             CREATED         STATUS
      PORTS                                                                 NAMES
```

Explanation:
This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ ./network.sh deployCC
\
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --la
ng node --label basic_1.0
+ res=0
```

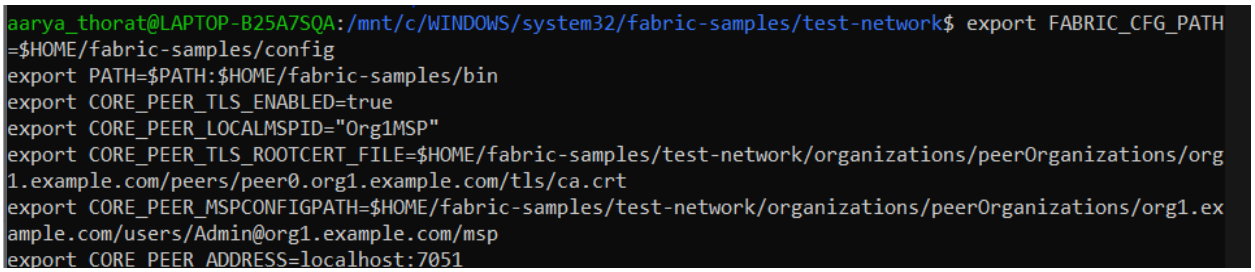
Explanation:
This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051



Explanation:
This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ cd /mnt/c/WINDOWS/syst
em32/fabric-samples/test-network

export PATH="$PWD/../bin:$PATH"
export FABRIC_CFG_PATH="$PWD/../config"

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export CORE_PEER_TLS_ROOTCERT_FILE="$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.or
g1.example.com/tls/ca.crt"

export CORE_PEER_MSPCONFIGPATH="$PWD/organizations/peerOrganizations/org1.example.com/users/Admin@org1.e
xample.com/msp"

export CORE_PEER_ADDRESS=localhost:7051

export ORDERER_CA="$(find "$PWD/organizations/ordererOrganizations/example.com" -name 'tlsca.example.com
-cert.pem' -print -quit)"
```

Explanation:
This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"Appra
isedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":
500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"
Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"
black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white
","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explanation:
This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Explanation:
This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ cd /mnt/c/WINDOWS/syst
em32/fabric-samples/test-network

export PATH="$PWD/../bin:$PATH"
export FABRIC_CFG_PATH="$PWD/../config"

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE="$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.or
g1.example.com/tls/ca.crt"
export CORE_PEER_MSPCONFIGPATH="$PWD/organizations/peerOrganizations/org1.example.com/users/Admin@org1.e
xample.com/msp"
export CORE_PEER_ADDRESS=localhost:7051

export ORDERER_CA="$(find "$PWD/organizations/ordererOrganizations/example.com" -name 'tlsca.example.com
-cert.pem' -print -quit)"
```

Explanation:
This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ peer chaincode invoke
\
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/t
ls/ca.crt" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/t
ls/ca.crt" \
-c '{"function":"TransferAsset","Args":["asset7","Aarya"]}'
2026-11-01 08:06:21.999 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successf
ul. result: status:200 payload:"Manav"
```

Explanation:
This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down

```
aarya_thorat@LAPTOP-B25A7SQA:/mnt/c/WINDOWS/system32/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] Running 11/11
 ✔Container peer0.org2.example.com       Removed                                          0.9s
 ✔Container orderer.example.com          Removed                                          0.5s
 ✔Container peer0.org1.example.com       Removed                                          0.9s
 ✔Volume compose_peer0.org3.example.com  Removed                                          0.0s
 ✔Volume compose_orderer3.example.com    Removed                                          0.0s
 ✔Volume compose_peer0.org2.example.com  Removed                                          0.0s
 ✔Volume compose_peer0.org1.example.com  Removed                                          0.0s
 ✔Volume compose_orderer2.example.com    Removed                                          0.0s
 ✔Volume compose_orderer4.example.com    Removed                                          0.0s
 ✔Volume compose_orderer.example.com     Removed                                          0.0s
 ✔Network fabric_test                    Removed                                          0.3s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d44557211281d889f87cd07bc2b5fbe664c87b24c015bee15cbcc7
```

Explanation:
This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.